Developing Interpretable Machine Learning for Forward Kinematics of Robotic Arms

Sai Teja Kanneganti
School of Computer Science
University of Oklahoma
Norman, OK, USA
kannegantisaiteja@ou.edu

Jin-Song Pei
School of Elec. & Comp. Engrg.
University of Oklahoma
Norman, OK, USA
jspei@ou.edu

Dean Frederick Hougen
School of Computer Science
University of Oklahoma
Norman, OK, USA
hougen@ou.edu

2021 IEEE Symposium Series on Computational Intelligence (SSCI) | 978-1-7281-9048-8/21/$31.00 ©2021 IEEE | DOI: 10.1109/SSCI50451.2021.9660074

***Abstract*—Machine learning (ML) is becoming increasingly sought after in diverse domains. Unfortunately for this objective, most ML research has focused on improving performance on evaluation metrics such as accuracy. However, to make important decisions, ML models need to be interpretable. We propose an approach to approximate kinematics of a robotic arm using interpretable artificial neural networks (ANNs). This work is based on approximating nonlinear functions where domain knowledge and visually observable features of the data are used to design ANNs. After analyzing existing work, we present a feasibility study approximating the kinematics of a simplified robotic arm and extend the work to multiple hidden layers. We generalize the existing work and extend its use for a different application, noting the challenges that arise while extending this work to multiple hidden layers.**

***Index Terms*—Interpretable Machine Learning, Trusted AI, Explainable AI, Fairness in AI, Robotics**

I. INTRODUCTION

Artificial neural networks (ANNs) are universal function approximators [1]. By having sufficiently many nodes in its hidden layer(s), any continuous nonlinear function can be learned by sigmoidal ANNs to any desired degree of accuracy. Unfortunately, standard ANNs are opaque because there is no inherent meaning to the hidden nodes. This makes it difficult to understand the model and restricts its use in making high-stake decisions. To bridge this knowledge gap for modeling kinematics, we use interpretable machine learning (IML), i.e., machine learning (ML) in which the response of the model given the input is inherently understandable [2]. We begin by recognizing existing IML research in nonlinear function approximation. This long-term multidisciplinary research effort has made strides toward providing end users of ANNs with interpretable modeling tools for engineered systems [3]–[7].

In this approach, ANNs are engineered as transparent, linear-in-the-weight parameterization tools by using domain knowledge/constraints and visual features to determine appropriate basis functions. With few hidden nodes to activate these basis functions, the modeling capability of sigmoidal ANNs is systematically revealed without relying on training. The key is the developed ANN initialization theory and practical procedures that can be interpreted via domain knowledge. A rich repository of direct (non-iterative) techniques and reusable ANN prototypes can be aggregated as the basis functions needed for specific problems, leading to interpretable ANNs as well as improved training performance and generalization as validated by simulated and real-world data [3]–[7].

Using forward kinematic equations, it is possible to describe the idealized location of a robot arm. But in practice there are problems with joint slippage, sag, and other mechanical issues. When greater weight is carried, the arm tends to sag more from the ideal position, potentially resulting in large discrepancies between what the idealized model predicts and what is observed in practice. As we deal with more complex robotic arms, it becomes more difficult to adequately model them using just equational models. For these reasons, it is appropriate to learn the model by trying different joint angles and measuring the location of the end effector. One approach is to build ML models of robotic arms. IML helps users to understand models and potentially provides better models.

In addition, robot kinematics may serve as a useful domain for developing IML theory by providing a scalable application problem. This is because robot kinematics allows us to start simple, then gradually add complexity in the number of input variables, the number of output variables, and the complexity of the relationships between them, thus provides a path for developing more complex ANNs.

This work is the first to use this approach to IML for robotics. We previously used this technique to model nonlinear dynamical systems in structural engineering [7] and epidemic curves [8]. The contributions of the present work are:

1) To show the feasibility of transferring the IML approach to a different application (robot kinematics).
2) To explore the feasibility and challenges of using this method to develop and train neural networks of up to two hidden layers and with multiple inputs.
3) To evaluate the performance of interpretable ANNs on different levels of noisy data.

The remainder of this paper is organized as follows: The target robotic arm and its forward kinematics are explained in Section II. An overview of the existing work and the theoretical foundation on which this research is built is presented in Section III. The experimental setup and the data used are explained in Section IV. The results and discussion are detailed in Section V and Section VI, respectively, before concluding in Section VII.

978-1-7281-9048-8/21/$31.00 ©2021 IEEE
Authorized licensed use limited to: Università degli Studi di Roma La Sapienza. Downloaded on July 30,2024 at 09:05:55 UTC from IEEE Xplore. Restrictions apply.

II. Domain Problem

A. Simple robot arm

Experiments were conducted using multiple nonlinear datasets based on a simplified model of a PUMA robot arm.

The target robot arm is a simple two-link robot arm with waist joint θ_1 that rotates 360 degrees in the horizontal plane; shoulder joint θ_2, that rotates 360 degrees vertically, with 0 degrees straight out to one side; and an elbow joint θ_3 that also rotates vertically, with 0 degrees bent up at a right angle to the first link. The elbow rotation can be from -90 to $+90$ degrees (considered the "elbow forward" configuration) or $+90$ to $+270$ degrees (considered the "elbow backward" configuration). Each link is $1/2$ unit length and the first link is situated at the origin. The arm is assumed to have zero volume which allows for full range of motion on all joints without the risk of self collisions.

Kinematics describes the relationship between a robot's joint coordinates and its spatial layout. It is a fundamental and classical topic in robotics. Kinematics calculations are used for tasks such as positioning a gripper at a position in space, designing a mechanism to move a tool between two points, and predicting if robot's motion would collide with obstacles. Kinematics is concerned with only the instantaneous values of robot's coordinates and is not concerned about the movements under forces and torques. In this work, we are only concerned with *forward kinematics*—using the kinematic equations to calculate the positions of the robot's links from specified values for the joints [9]. In our experiments, we design models for predicting the end position in X, Y, and Z three-space given normalized waist ($\hat{\theta}_1$), shoulder ($\hat{\theta}_2$), and elbow ($\hat{\theta}_3$) angles.

B. Equations

The kinematics equations for the series chain of a robot with revolute or prismatic joints are obtained using a rigid transformation $[J_i]$ to characterize the relative movement allowed at each joint and a separate rigid transformation $[K_i]$ to define the dimensions of each link. The result is a sequence of alternating joint and link transformations from the base to the end link,

$$[T] = [J_1][K_1][J_2][K_2]\ldots[J_n][K_n] \tag{1}$$

where $[T]$ is the transformation locating the end-link. The equations for the forward kinematics of our simplified arm are obtained using (1) and can be seen in Table I.

III. Interpretable Machine Learning

Traditional ML algorithms are opaque. IML is transparent as it injects domain knowledge into ML design. This results in a system that is understandable and trustworthy. Early IML research for pattern classification using ANNs states, "The first layer partitions the input space into a number of cells. The sole function of additional layers is then to group these cells into decision regions" [10]. This fundamental idea can also be found in deep learning [11] and explainable AI (XAI) [12]. This fundamental idea is the basis of designing transparent sigmoidal neural networks for function approximation.

TABLE I
Equations for different input-output combinations derived using forward kinematics of robotic arm.

Input	Output	Equation
$\hat{\theta}_1$	$\hat{x}$	$0.5\cos(2\pi\hat{\theta}_1) + 0.5$
$\hat{\theta}_1$	$\hat{y}$	$0.5\sin(2\pi\hat{\theta}_1) + 0.5$
$\hat{\theta}_1$	$\hat{z}$	0.5
$\hat{\theta}_2$	$\hat{x}$	$0.5\cos(2\pi\hat{\theta}_2 + \pi/4) + 0.5$
$\hat{\theta}_2$	$\hat{y}$	0
$\hat{\theta}_2$	$\hat{z}$	$0.5\sin(2\pi\hat{\theta}_2 + \pi/4) + 0.5$
$\hat{\theta}_3$	$\hat{x}$	$-0.5\sin(2\pi\hat{\theta}_3) + 0.5$
$\hat{\theta}_3$	$\hat{y}$	0
$\hat{\theta}_3$	$\hat{z}$	$0.5\cos(2\pi\hat{\theta}_3) + 0.5$
$\hat{\theta}_1$, $\hat{\theta}_2$	$\hat{x}$	$0.5[\cos(2\pi\hat{\theta}_1)\cos(2\pi\hat{\theta}_2 + \pi/4) + 1]$
$\hat{\theta}_1$, $\hat{\theta}_2$	$\hat{y}$	$0.5[\sin(2\pi\hat{\theta}_1)\cos(2\pi\hat{\theta}_2 + \pi/4) + 1]$
$\hat{\theta}_1$, $\hat{\theta}_2$	$\hat{z}$	$0.5\sin(2\pi\hat{\theta}_2 + \pi/4) + 0.5$

In the work reviewed here, static nonlinear functions are approximated using feedforward ANNs, and using sigmoidal activation functions with a single hidden layer, and taking an input vector to produce a scalar output. To design interpretable neural networks, we use domain knowledge including visually observable features to determine (a) structure of the network (including the number of hidden nodes) (b) initial values of the parameters (weights and biases in the network) as the first approximation of the function to be learned, and then trained on the data for local optimization. These initial parameters are obtained based on theory and not from training. Determining structure and initial values of ANN is a longstanding question [13] that remains open in general.

Structure, parameter values, and error bounds to carry out basic computing operations and polynomial functions with certain approximation accuracy have been derived in earlier work, including the number of hidden nodes, weights, and biases for approximating four basic arithmetic operations [14] and the number of hidden nodes, weights, and biases for approximating polynomial terms p^0, p^1, p^2, and p^3 [4].

Here is an example to demonstrate approximating a constant term ($p^0 = 1$) using two sigmoidal units (S) where

$$S(x) = \frac{1}{1 + e^{-x}}. \tag{2}$$

In Fig. 1, W_1 is the array of weights from the input layer to the hidden layer and W_2 is the weight array from the hidden layer to the output layer. Likewise, b_1 is the bias array of the hidden nodes and b_2 is the bias of the output node.

W_1 is $[1, -1]$, and W_2 has the value of constant we would like to approximate, so W_2 becomes $[1, 1]$, b_1 takes the values of $[0, 0]$, and b_2 equals [0]. Considering p as the input, the output takes the value of $1 \times (S(p \times 1 + 0)) + 1 \times (S(p \times -1 + 0)) + 0$. Solving this gives an output of 1.

This approach can be used to approximate any constant value. Thus, IML provides a meaningful interpretation of nodes used in the ANNs. In this work, *custom initialization* refers to the IML parameter determination approach explained in this section.

Authorized licensed use limited to: Universita degli Studi di Roma La Sapienza. Downloaded on July 30,2024 at 09:05:55 UTC from IEEE Xplore. Restrictions apply.

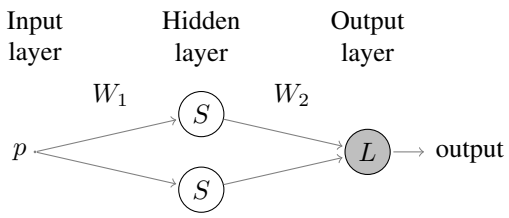

Fig. 1. ANN architecture used to approximate a constant term. Nodes labeled S and L have sigmoidal and linear activation functions, respectively.

Domain knowledge provides useful information. First, we try to answer these questions: (a) What are the useful relations between input and output? This can be understood by visually observing the features of the data. (b) What sigmoidal neural network prototypes can be used for each useful feature? Prototypes and their variants are predetermined ANNs that are constructed in advance from a forward formulation based on either algebraic or geometric capabilities of linear sums of sigmoidal functions to capture some specific/dominant features of the nonlinear function to be approximated [5]. We match each dominant feature to a prototype before concatenating them within one or more hidden layers for backpropagation to be applied. This technique captures important features and discloses more inner-workings through visualization, theoretical, and mathematical work.

IV. EXPERIMENTS

AN objective of our work is to see the feasibility of designing interpretable ANN models for a simplified robotic arm. Apart from interpretability, we hypothesized that our interpretable custom ANNs would have training success and good generalization capabilities. We are also interested analyzing the performance when varying the number of input features. In this work we have used one and two input features to predict a target variable. This helps us analyze the complexities and performance involved while training with more input features. Discussed below are data generation, experimental setup, and initial parameters for the custom approach used in this paper.

A. Data Generation

The data generated from the kinematics equations does not contain any noise. We added different levels of noise to the data to provide a range of test conditions in the experiments. In robotics, there is a chance for two different types of noise—motor and sensor. Because motor noise tends to be more problematic in practice, we generated synthetic motor noise.

Synthetic motor noise is obtained by adding noise after the input. For each input-output combination, we generate data at four different noise levels—no, low, medium, and high noise. Low, medium, and high noise data have zero-mean noise with a standard deviation of 5, 10, and 15 degrees, respectively. Using the equations in Table I, data is generated by adding noise to the input angles before computing the output, resulting in noisy output while retaining the original (non-noisy) value for each input.

The noise in the high noise case is greater than is generally observed in real-world use, although if arms were used nearer to their physical carrying capacity, such noise might be seen.

In our kinematics we have three input angles (θ_1, θ_2, and θ_3) that determine the end position (x, y, and z) of the robot end effector. When we experiment with one input, we are interested in learning the input-output relation w.r.t. that single input. Then we vary that input alone by keeping all the remaining inputs constant at 0. When we experiment with two inputs, we are interested in learning the output relation w.r.t. these two inputs. Then we vary only those two inputs. While modeling, varying inputs are treated as inputs to a neural network.

For each input-output combination, we have generated 4,000 samples of data for each category of noise. All the input angles are randomly generated covering the entire input range. Based on the noise levels, noise is randomly added to the data. While training each model, 3,000 samples of this data are used for training, 500 samples are used for validation, and the remaining 500 samples are used for testing.

Input angles vary from 0 to 360 degrees (0 to 2π radians) and are normalized between 0 and 1. Normalizing inputs is common practice in ML as it speeds convergence. For every input combination (varying inputs), we obtained possible minimum and maximum values for each output. These minimum and maximum values are used as lower and upper bound to normalize each output variable. By normalizing outputs, error reported will be uniform across all input-output combinations and thus helps for easy and fair comparison. Thus, inputs ($\hat{\theta}_1$, $\hat{\theta}_2$, and $\hat{\theta}_3$) and outputs ($\hat{x}$, $\hat{y}$, and $\hat{z}$) are scaled between 0 and 1. This scaled data is used for training the ANNs.

B. Experimental design

To compare with our custom initialization, we use Nguyen-Widrow (NW) initialization, in which weights are assigned in such a manner that the region of interest is divided into small intervals. The training process speeds up by setting the initial weights of the hidden layer, so that each hidden node is assigned its own interval at the start of training. The network is trained with each hidden node still having the freedom to adjust its interval size and location during the training. Thus, the NW approach generates initial weights and bias values for a layer such that the active regions of the layer's neurons are distributed approximately evenly over the input space [15].

The Adam optimizer [16] is used for training these ANNs. For shallow neural networks, we have observed that the Levenberg–Marquardt optimizer [17] performs better than Adam. However, we used Adam here as we are interested in continuing on to train the deeper neural networks that will be engineered when we add more inputs. Levenberg–Marquardt is not feasible for training deep neural networks.

A learning rate of 0.007 was chosen based on preliminary experimentation. We used a mini-batch size of 32 and trained models for $1,500$ epochs. When mean squared error (MSE) is used as the loss function, the optimizer tries to reduce the square of the error, which magnifies the penalties of larger errors as compared to those of smaller errors. Based

Authorized licensed use limited to: Universita degli Studi di Roma La Sapienza. Downloaded on July 30,2024 at 09:05:55 UTC from IEEE Xplore. Restrictions apply.

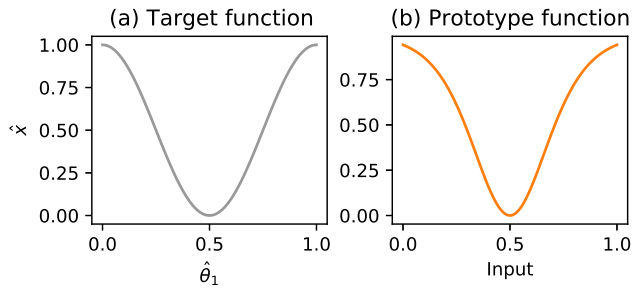

Fig. 2. $\hat{x}$ to be approximated given $\hat{\theta}_1$ (a) Target function to be approximated (b) Prototype function obtained using IML to approximate target function.

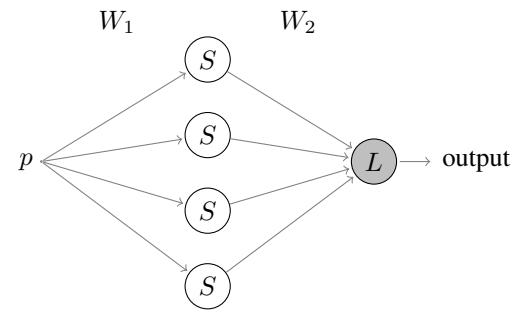

Fig. 3. ANN architecture with one hidden layer and four hidden nodes. S, L nodes correspond to sigmoidal, linear activation functions respectively.

on the applications of robotic arms, it makes sense to more substantially penalize larger errors. Thus, the loss and the evaluation metric used are both MSE.

For each dataset, we do 30 repetitions to collect statistically meaningful results. For each repetition, data is randomly split between training, validation, and test. Models are trained with initial parameters and their performance is recorded.

To compare results, the Mann–Whitney U test is chosen because it is a nonparametric test used to compare two distributions of independent samples. Nonparametric methods allow statistical inferences without making the assumption that the samples have been taken from a particular distribution. The Mann–Whitney U test is a commonly used alternative to two sample t-test when the distribution is non-normal [18].

C. Initial parameters for custom approach

When approximating equations in Table I, we do not approximate constant values; approximating a constant does not require any nonlinear approximation and can be approximated using two hidden nodes as described in Section III. Moreover, $\hat{\theta}_3$ as input and $\hat{z}$ as output, takes the same form as $\hat{\theta}_1$ as input and $\hat{x}$ as output. $\hat{\theta}_1$,$\hat{\theta}_2$ as inputs and $\hat{z}$ as output also takes the same form as $\hat{\theta}_2$ as input and $\hat{z}$ as output. So, we will not be approximating for these input-output combinations.

1) $\hat{x} = 0.5\cos(2\pi\hat{\theta}_1)+0.5$: Fig. 2(a) shows the function to be approximated. Dominant features including visually observable features in the data help us to design the architecture and initial values of neural network. This function closely takes the shape of the prototype function in Fig. 11(b) of [14], where the first order partial derivative of a sigmoidal function with respect to bias is observed

$$\frac{\partial S(W,p,b)}{\partial b} = \frac{e^{-(Wp+b)}}{(1+e^{-(Wp+b)})^2} \tag{3}$$

and is plotted using $W = 7$ and $b = 0$, where W and b are the incoming weight and bias of the neuron. This prototype function in (3) can be approximated using linear sum of two sigmoidal functions [8] and the equation is as follows

$$\frac{\partial S(W,p,b)}{\partial b} \approx \frac{1}{2\Delta b}\big(S(W,p,b+\Delta b)-S(W,p,b-\Delta b)\big) \tag{4}$$

Equation (4) can be approximated using two hidden nodes, as shown in Fig. 1. We choose a small value of Δb equals to 0.1, it is an empirical choice and needs to be a small value. Based on the above information, W_1 are [7, 7]. From (4), we can observe that W_2 are $[\frac{1}{2\Delta b}, -\frac{1}{2\Delta b}]$ respectively. Thus, W_2 becomes [5, -5]. b_1 takes the values of $[b+\Delta b,\ b-\Delta b]$, b_1 equals [0.1, -0,1], and b_2 equals [0].

Initial weights to approximate are obtained in three steps. First we scale the prototype function to our required input and output range. Our required input and output range is from 0 to 1 and can be observed in Fig. 2(a). The input and output range for the prototype is from -1 to 1 and 0 to 0.25 respectively. The prototype function can be made closer to Fig. 2(a), by only considering the input range between [-0.6 to 0.6], the weights and bias of scaled function can be obtained by substituting values of $\hat{\theta}_1$ = 0 for p = -0.6 and $\hat{\theta}_1$ = 0.5 for p = 0. Now, W_1 changes to [8.4, 8.4] and b_1 changes to [-4.1, -4.3]. To scale the output between 0 and 1, W_2 are multiplied by 4, thus W_2 changes to [20, -20]. Now these weights are better used for our input and output ranges.

In second step, we flip the function by changing the sign of the output. Thereby bringing the prototype function more closer to function in Fig. 2(a). Sign of the output can be reversed by changing the sign of the final layer weights, thus W_2 becomes [-20, 20].

In final step, we add a constant of 1, to bring this function more closer to the function in Fig. 2(a). To add a constant we need two hidden nodes as described in Section III. Now W_1 corresponding to these two hidden nodes are [1, -1], b_1 corresponding to these two hidden nodes are [0, 0], and W_2 corresponding to these two hidden nodes are [1, 1]. Now, we have a total of 4 hidden nodes, 2 from earlier prototype function and 2 from the the addition of constant. Thus our final initial parameters are: W_1 are [8.4, 8.4, 1, -1], b_1 are [-4.1, -4.3, 0, 0], W_2 are [-20, 20, 1, 1], and b_2 equals [0]. Now these weights are used to approximate function in Fig. 2(b). This function is very close to the function we are interested in approximating, i.e., Fig. 2(a). These parameters takes the shape of ANN architecture in Fig. 3.

2) $\hat{y} = 0.5\sin(2\pi\hat{\theta}_1) + 0.5$: Fig. 4(a) shows the function to be approximated. This function closely takes the shape of the prototype 3, variant c in Fig. 3 of [5]. To approximate this prototype function we require two hidden nodes, similar to Fig. 1. The initial parameters to approximate this function

Authorized licensed use limited to: Universita degli Studi di Roma La Sapienza. Downloaded on July 30,2024 at 09:05:55 UTC from IEEE Xplore. Restrictions apply.

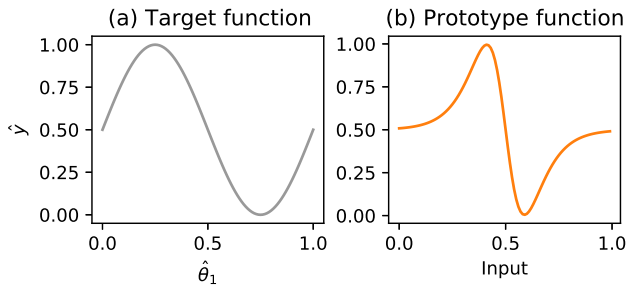

Fig. 4. $\hat{y}$ to be approximated given $\hat{\theta}_1$ (a) Target function to be approximated (b) Prototype function obtained using IML to approximate target function.

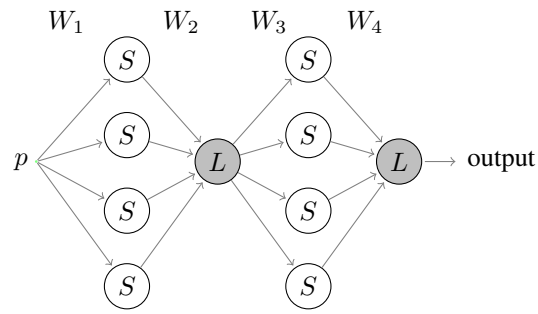

Fig. 5. ANN architecture before condensing hidden layer. S, L nodes correspond to sigmoidal, linear activation functions respectively.

follows, W_1 are [20, 10], W_2 are [1, -1]. b_1 takes the values of [0, 0] and b_2 equals to [0]. These initial values can be found in Table. 1 of [5].

Similar to above equation, initial weights to approximate are obtained in three steps. First we scale the prototype function to our required input and output range. Our required input and output range is from 0 to 1. The input and output range for the prototype is from -1 to 1 and -0.15 to 0.15 respectively. The prototype function can be made closer to Fig. 4(a), by only considering the input range between [-0.6 to 0.6]; the weights and bias of scaled function can be obtained by substituting values of $\hat{\theta}_1$ = 0 for p = -0.6 and $\hat{\theta}_1$ = 0.5 for p = 0. Now, W_1 changes to [24, 12] and b_1 changes to [-12, -6]. To scale the output between 0 and 1, W_2 are multiplied by 3.3, thus W_2 changes to [3.3, -3.3]. Now these weights are better used for our input and output ranges.

In second step, we flip the function by changing the sign of the output. Thereby bringing the prototype function more closer to function in Fig. 4(a). Thus W_2 becomes [-3.3, 3.3]. In final step, we add a constant of 0.5. To add a constant we need two hidden nodes. Now W_1 corresponding to these two hidden nodes are [1, -1], b_1 corresponding to these two hidden nodes are [0, 0], and W_2 corresponding to these two hidden nodes are [0.5, 0.5]. Now, we have a total of 4 hidden nodes, 2 from earlier prototype function and 2 from the the addition of a constant. Thus our final initial parameters are: W_1 are [24, 12, 1, -1], b_1 are [-12, -6, 0, 0], W_2 are [-3.3, 3.3, 0.5, 0.5], and b_2 = [0]. Now these weights are used to approximate function in Fig. 4(b). This function is very close to the function we are interested in approximating, i.e., Fig. 4(a). These parameters takes the shape of ANN architecture in Fig. 3.

3) $\hat{x} = 0.5\cos(2\pi\hat{\theta}_2 + \pi/4) + 0.5$*:* This equation is similar to equation in Section IV-C1, except it has a positive phase shift of $\pi/4$. Approximating this equation involves two steps. First, we approximate $2\pi\hat{\theta}_2 + \pi/4$ and then feed this as an input to neural network with initial values as explained in Section IV-C1. To approximate $2\pi\hat{\theta}_2 + \pi/4$ we need four hidden nodes, two for approximating $2\pi\hat{\theta}_2$ (first order polynomial term) explained in [4] and two for approximating $\pi/4$ (constant term). To approximate $2\pi\hat{\theta}_2$, W_1 are [0.1, -0.1], b_1 are [0, 0], W_2 are [40π, -40π], and b_2 equals [0]. To approximate $\pi/4$, W_1 are [1, -1], b_1 are [0, 0], W_2 are [$\pi/4$, $\pi/4$], and b_2 = [0].

Thus to approximate $2\pi\hat{\theta}_2 + \pi/4$ the initial values used are, W_1 are [0.1, -0.1, 1, -1], b_1 are [0, 0, 0, 0], W_2 are [40π, -40π, $\pi/4$, $\pi/4$], and b_2 = [0]. Now, we concatenate the neural network with initial values as explained in Section IV-C1. Thus, W_3 are [8.4, 8.4, 1, -1], b_3 are [-4.1, -4.3, 0, 0], W_4 are [-20, 20, 1, 1], and b_4 = [0]. This takes the shape of neural network architecture in Fig. 5. This neural network can be condensed to neural network in Fig. 6, by removing the hidden layer with nodes having linear activation. Hidden layer can be removed by performing matrix multiplication of incoming and outgoing weights of that layer [14].

So, the new weights formed is a product of W_2 and W_3. This causes new weights to explode and results in unstable learning. More about this is discussed in Section VI. To avoid this, W_2 is balanced, i.e., larger weights are made smaller by trying to preserve the ratio of weights. So, 40π and -40π in W_2 changes to $\pi/4$ and -$\pi/4$. Thus, W_2 changes to [$\pi/4$, -$\pi/4$, $\pi/4$, $\pi/4$]. Thus $W_2 \times W_3$ becomes [[2.1π, 2.1π, $\pi/4$, -$\pi/4$], [-2.1π, -2.1π, -$\pi/4$, $\pi/4$], [2.1π, 2.1π, $\pi/4$, -$\pi/4$], [2.1π, 2.1π, $\pi/4$, -$\pi/4$]]. Here, first sub list ([2.1π, 2.1π, $\pi/4$, -$\pi/4$]) corresponds to outgoing weights from first hidden node of first hidden layer to all the four hidden nodes of second hidden layer respectively. Similarly, second sub list ([-2.1π, -2.1π, -$\pi/4$, $\pi/4$]) corresponds to outgoing weights from second hidden node of first hidden layer to all the four hidden nodes of second hidden layer respectively. After condensation, bias values to the nodes remain unchanged. The results shown in Table II are obtained after condensing a hidden layer with linear activation units.

4) $\hat{z} = 0.5\sin(2\pi\hat{\theta}_2 + \pi/4) + 0.5$*:* This equation is similar to equation in Section IV-C2, and has a positive phase shift of $\pi/4$. Approximating this equation involves two steps. This equation is approximated similar to Section IV-C3. First, we approximate $2\pi\hat{\theta}_2 + \pi/4$ using four hidden nodes and then feed this as an input to neural network with initial values as explained in Section IV-C2. This also takes the shape of neural network architecture in Fig. 6. Thus the initial values becomes, W_1 are [0.1, -0.1, 1, -1], b_1 are [0, 0, 0, 0], $W_2 \times W_3$ becomes [[6π, 3π, $\pi/4$, -$\pi/4$], [-6π, -3π, -$\pi/4$, $\pi/4$], [6π, 3π, $\pi/4$, -$\pi/4$], [6π, 3π, $\pi/4$, -$\pi/4$]]. Bias to the hidden units equals [-12, -6, 0, 0], W_4 are [-3.3, 3.3, 0.5, 0.5], and bias to the

Authorized licensed use limited to: Universita degli Studi di Roma La Sapienza. Downloaded on July 30,2024 at 09:05:55 UTC from IEEE Xplore. Restrictions apply.

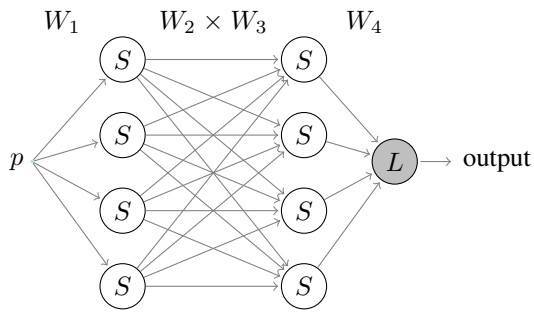

Fig. 6. ANN architecture after condensing hidden layer. S, L nodes correspond to sigmoidal, linear activation functions respectively.

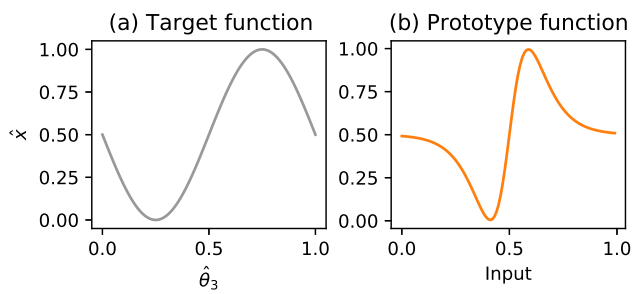

Fig. 7. $\hat{x}$ to be approximated given $\hat{\theta}_3$ (a) Target function to be approximated (b) Prototype function obtained using IML to approximate target function.

output layer equals [0].

5) $\hat{x} = -0.5\sin(2\pi\hat{\theta}_3) + 0.5$: Fig. 7(a) shows the function to be approximated. This function closely takes the shape of function approximated in Section IV-C2. Initial weights can be obtained as similar to Section IV-C2. First, input and output values of prototype are scaled. Here we need not change the sign of the output. Later, we add a constant of 0.5. So, our final initial parameters are: W_1 are [24, 12, 1, -1], b_1 are [-12, -6, 0, 0], W_2 are [3.3, -3.3, 0.5, 0.5], and b_2 = [0]. Now these weights are used to approximate function in Fig. 7(b). These parameters takes the shape of ANN architecture in Fig. 3.

6) $\hat{x} = 0.5[\cos(2\pi\hat{\theta}_1)\cos(2\pi\hat{\theta}_2+\pi/4)+1]$: As discussed in Section IV-C1, two hidden nodes can be used to approximate $\cos(2\pi\hat{\theta}_1)$ and these two nodes require $\hat{\theta}_1$ as input. Similarly, two hidden nodes can be used to approximate $\cos(2\pi\hat{\theta}_2+\pi/4)$ and these two nodes require $\hat{\theta}_2$ as input. The later has a phase shift of $\pi/4$, so, $\cos(2\pi\hat{\theta}_2 + \pi/4)$ can still be approximated by using either $\cos(2\pi\hat{\theta}_2)$ or $\sin(2\pi\hat{\theta}_2)$. The same architecture is capable of learning phase shifts in the input. We choose to approximate using $\sin$ prototype. Fig. 8 shows the architecture used to approximate this equation. First two hidden nodes in the first hidden layer approximates $\cos(2\pi\hat{\theta}_1)$ and next two hidden nodes in that layer approximates $\cos(2\pi\hat{\theta}_2+\pi/4)$. The missing connections between inputs and first hidden layer has an initial value of 0. So, W_1 are [[8.4, 8.4, 0, 0], [0, 0, 24, 12]], $b1$ are [-4.1, -4.3, -12, -6], and W_2 are [[-20, 0], [20, 0], [0, -3.3], [0, 3.3]]. The nonzero weights corresponding to first two hidden nodes are the same weights derived after step 2 in Section IV-C1. Similarly, the nonzero weights corresponding

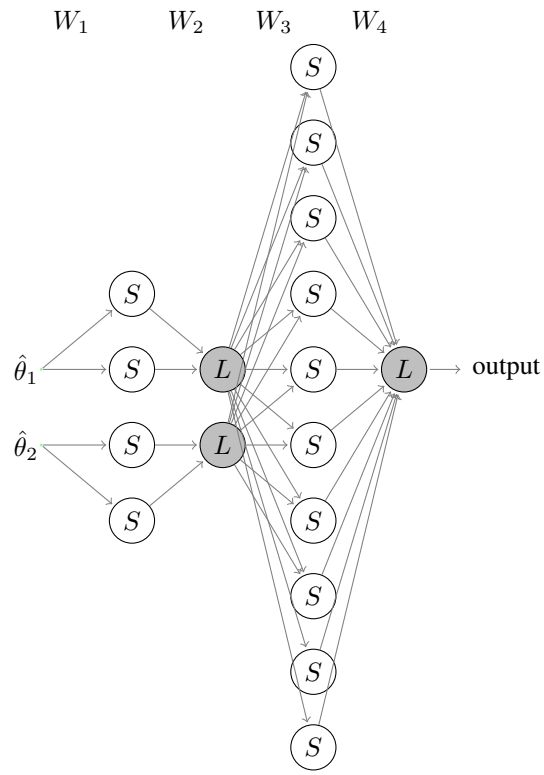

Fig. 8. ANN architecture before condensing hidden layer - for $\hat{\theta}_1, \hat{\theta}_2$ as inputs. S, L nodes correspond to sigmoidal, linear activation functions respectively.

to last two hidden nodes are the same weights derived after step 2 in Section IV-C2.

Approximating two first order polynomial terms requires 8 nodes and it's initial values are mentioned in Table 2 of [14]. So, $\cos(2\pi\hat{\theta}_1)\times\cos(2\pi\hat{\theta}_2+\pi/4)$ can be approximated using 8 hidden nodes. Two hidden nodes can be used to add a constant of 1. To approximate a constant term, the last two hidden nodes in third hidden layer are connected to first node in second hidden layer. So, W_3 are [[0.1, 0.1], [-0.1, -0.1], [1, 1], [-1, -1], [0.1, -0.1], [-0.1, 0.1], [1, -1], [-1, 1], [1, 0], [-1, 0]], b_3 are [-10, -10, 0, 0, -10, -10, 0, 0, 0, 0]. W_4 are [5.507e5, 5.507e5, -50.006, -50.006, -5.507e5, -5.507e5, 50.006, 50.006, 1, 1]. W_4 are the final weights, these weights needs to be further divided by 2 as the whole equation is divided by 2 and b_4 = [0]. This weights and architecture in Fig. 8 can be condensed to Fig. 9 by removing the hidden layer with nodes having linear activation function [14].

7) $\hat{y} = 0.5[\sin(2\pi\hat{\theta}_1)\cos(2\pi\hat{\theta}_2+\pi/4)+1]$: This equation is similar to equation in Section IV-C6, the only thing that differs is $\sin(2\pi\hat{\theta}_1)$ instead of $\cos(2\pi\hat{\theta}_1)$. So, the nonzero weights corresponding to these hidden nodes (first two hidden nodes of first hidden layer) are the same weights derived after step 2 in Section IV-C2. So, W_1 are [[24, 12, 0, 0], [0, 0, 24, 12]], $b1$ are [-12, -6, -12, -6], and W_2 are [[-3.3, 0], [3.3, 0], [0, -3.3], [0, 3.3]]. And later layer weights and bias are same as those mentioned in Section IV-C6.

Authorized licensed use limited to: Universita degli Studi di Roma La Sapienza. Downloaded on July 30,2024 at 09:05:55 UTC from IEEE Xplore. Restrictions apply.

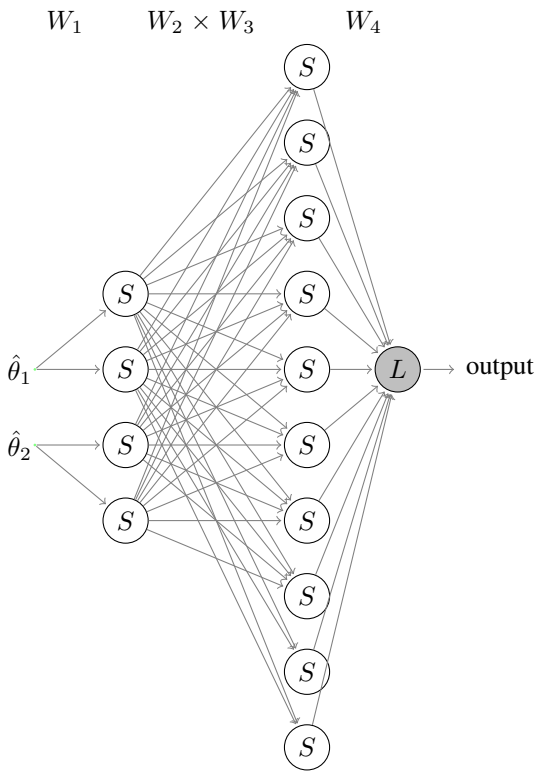

Fig. 9. ANN architecture after condensing hidden layer - for $\hat{\theta}_1, \hat{\theta}_2$ as inputs. S, L nodes correspond to sigmoidal, linear activation functions respectively.

D. Tools and software used

All our experiments are executed in TensorFlow version 2.5.0 using Python version 3.7.10. Initial weights of neural networks with NW approach are obtained from MATLAB version 9.6.

V. RESULTS

Results in Table II are obtained when models are trained on training data and evaluated on test data. Values in "Best Error" column of Table II suggests that the architectures recommended by IML approach is capable of approximating the functions. Fig. 10(a), Fig. 10(b), and Fig. 10(c) are prototypical examples of learning excellently, learning moderately and learning poorly respectively. Actual values in Fig. 10 are the values obtained using equations in Table II, and these values correspond to 0 error. In Fig. 10(a), models from both the approaches learned very well to approximate the function over the entire region of input. In Fig. 10(b), models from both the approaches moderately learned to approximate function. We can observe that these models did not precisely learn for some input range. NW initialization approach in Fig. 10(c) has learned poorly. This is the result of the optimizer stuck at local optima. These figures correspond to training on no noise data.

VI. DISCUSSION

There are two components to the IML approach stated above. The first component is determining the structure of the

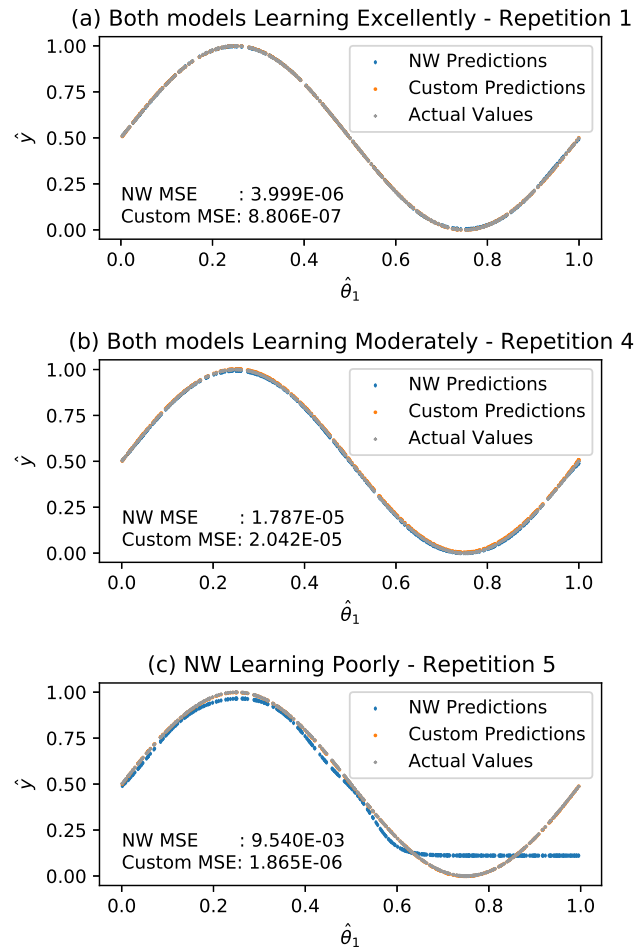

Fig. 10. Prototypical example of learning a function (a) Excellent learning (b) Moderate learning (c) NW Learning poorly

neural network. From Table II, we can observe that the number of layers and nodes recommended by the IML approach is capable of learning the functions. The second component is determining the initial values of the neural network. While approximating some functions, Table II strongly indicates that our custom constructive initialization performs consistently and converge to reasonable accuracy for all the repetitions. However, from Table II, Fig. 11(a), and Fig. 11(b) we can observe that NW initialization results in model stuck at local optima for one or more repetitions. This is also observed when NW outperforms custom initialization in many repetitions, this can be observed for 13th repetition in Fig. 11(b). Fig. 12 shows the progression of error during the training, it is a prototypical example of NW being stuck at local optima. NW does not have just one initial values but rather has multiple initial values. We observed that one or more of these initial values are resulting the model to stuck at local optima. Besides interpretability, IML models also help minimize the error; it is suggested to model using both approaches as IML approach helps validate if the NW model got stuck at local optima.

For $\hat{\theta}_2$ as the input, we have balanced the weights to control

Authorized licensed use limited to: Universita degli Studi di Roma La Sapienza. Downloaded on July 30,2024 at 09:05:55 UTC from IEEE Xplore. Restrictions apply.

TABLE II

COMPARISON OF MODELS WITH TWO DIFFERENT INITIALIZATIONS. FOR NOISE LEVEL, N, L, M, AND H CORRESPOND TO NO, LOW, MEDIUM, AND HIGH NOISE, RESPECTIVELY. BEST ERROR IS THE SMALLEST ERROR REPORTED IN ANY REPETITION FROM EITHER APPROACH BASED ON THE ARCHITECTURE SUGGESTED BY THE IML APPROACH. M-W p-VALUE IS THE VALUE OF THE MANN-WHITNEY U TEST.

Input	Output	Noise Level	Best Error	M-W p-value	Mean MSE		Median MSE		Repetition Count		
					NW	Custom	NW	Custom	Lower Custom Error	NW Local Optima	Custom Local Optima
$\hat{\theta}_1$	$\hat{x}$	N	7.964e-07	3.019e-11	**5.042e-06**	5.085e-05	**4.330e-06**	4.299e-05	0	0	0
		L	8.753e-04	0.002	**1.021e-03**	1.141e-03	**1.000e-03**	1.091e-03	4	0	0
		M	3.411e-03	0.185	**3.919e-03**	4.074e-03	**3.901e-03**	4.036e-03	9	0	0
		H	7.363e-03	0.245	**8.579e-03**	8.850e-03	**8.579e-03**	8.618e-03	10	0	0
	$\hat{y}$	N	8.303e-07	1.336e-05	2.018e-03	**4.604e-06**	6.180e-06	**1.849e-06**	24	7	0
		L	8.699e-04	0.108	2.973e-03	**1.053e-03**	1.107e-03	**1.058e-03**	19	7	0
		M	3.416e-03	0.245	5.255e-03	**4.141e-03**	4.243e-03	**4.130e-03**	19	4	0
		H	7.553e-03	0.641	9.827e-03	**9.002e-03**	9.030e-03	**8.954e-03**	16	4	0
$\hat{\theta}_2$	$\hat{x}$	N	2.873e-07	5.572e-10	**1.891e-05**	6.866e-05	**9.755e-07**	5.219e-05	1	1	0
		L	9.397e-04	2.278e-05	**1.091e-03**	1.338e-03	**1.063e-03**	1.232e-03	4	0	0
		M	3.299e-03	6.097e-03	**4.080e-03**	4.395e-03	**4.002e-03**	4.362e-03	3	0	0
		H	7.328e-03	0.030	**8.827e-03**	9.183e-03	**8.701e-03**	9.083e-03	6	0	0
	$\hat{z}$	N	4.280e-07	0.009	1.382e-05	**8.440e-06**	**1.500e-06**	3.016e-06	9	1	0
		L	8.422e-04	0.162	**1.028e-03**	1.081e-03	**1.020e-03**	1.070e-03	10	0	0
		M	3.300e-03	0.620	**3.914e-03**	3.979e-03	**3.869e-03**	3.887e-03	10	0	0
		H	7.207e-03	0.491	**8.510e-03**	8.594e-03	**8.265e-03**	8.609e-03	10	0	0
$\hat{\theta}_3$	$\hat{x}$	N	1.312e-06	3.988e-4	8.171e-04	**5.566e-06**	5.789e-07	**2.887e-05**	22	2	0
		L	8.610e-04	0.853	1.869e-03	**1.069e-03**	**1.052e-03**	1.072e-03	16	2	0
		M	3.433e-03	0.994	4.862e-03	**4.114e-03**	**4.062e-03**	4.098e-03	18	2	0
		H	7.487e-03	0.717	9.513e-03	**8.728e-03**	**8.619e-03**	8.790e-03	18	2	0
$\hat{\theta}_1, \hat{\theta}_2$	$\hat{x}$	N	2.650e-05	1.247e-4	**2.058e-04**	2.947e-04	**1.004e-04**	2.672e-04	5	0	0
		L	1.069e-03	1.370e-03	**1.346e-03**	1.537e-03	**1.290e-03**	1.481e-03	5	0	0
		M	3.299e-03	0.004	**4.438e-03**	4.748e-03	**4.413e-03**	4.693e-03	8	0	0
		H	8.003e-03	0.864	**9.482e-03**	9.484e-03	9.302e-03	**9.267e-03**	14	0	0
	$\hat{y}$	N	3.558e-05	2.033e-09	**2.192e-04**	1.007e-03	**1.219e-04**	8.345e-04	1	0	0
		L	8.935e-04	8.890e-10	**1.352e-03**	2.475e-03	**1.294e-03**	2.205e-03	1	0	0
		M	3.555e-03	5.091e-06	**4.387e-03**	5.449e-03	**4.353e-03**	5.057e-03	4	0	0
		H	7.203e-03	3.324e-06	**8.922e-03**	1.051e-02	**8.868e-03**	9.989e-03	4	0	0

the weights from exploding. When weights are not balanced, model is unstable and it is unlearning what it has partially learned. Fig. 13 is a prototypical example of model trained using custom initialization is unstable but model trained using NW initialization has learned the function.

IML encourages sparse learning. The approach mentioned in this paper uses less number of nodes to approximate a target function, thus avoids overfitting in ANNs. We have demonstrated one capable custom model for each target function. There can be many such custom models, what we presented is merely one initialization, not necessarily the most optimal in any sense [5]. One limitation to this IML approach is that it cannot be used to approximate functions or systems involving transient state responses.

VII. CONCLUSIONS AND FUTURE WORK

This paper explains benefits to robot kinematics as an application to IML. In this work we have shown the feasibility of IML for a different application. Errors reported in Table II also suggests that the number of hidden layers and nodes recommended by IML approach is capable of learning the functions. Bias in our approach is working on informed functions rather than not knowing the function to approximate. IML emphasizes using domain knowledge and constraints to develop ML. If domain knowledge is used for neural network initialization, then training would more probably converge to the global optima rather than just a local optima, making the trained neural network more consistent, accurate, and meaningful, hence helps roboticists make informed decisions.

In this work, we have demonstrated one capable custom model (IML initial values) for each target function. In the future, we will experiment and analyze the performance of other capable custom models. Earlier, this IML approach was applied to a different domain. We have adapted the IML approach for robot kinematics and used two hidden layers to approximate some of the functions. We intend to test and extend its use for more deeper neural networks. Apart from more layers, we are interested to extend this work for

Authorized licensed use limited to: Universita degli Studi di Roma La Sapienza. Downloaded on July 30,2024 at 09:05:55 UTC from IEEE Xplore. Restrictions apply.

Fig. 11. Errors reported while training across all repetitions (a) $\hat{y}$ given $\hat{\theta}_1$ (b) $\hat{x}$ given $\hat{\theta}_2$

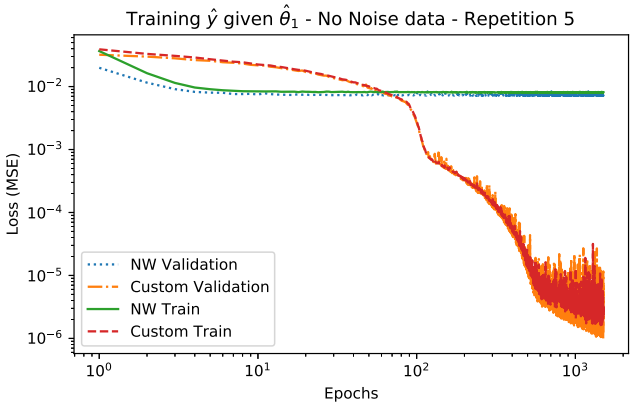

Fig. 12. History of errors when NW stuck at local optima

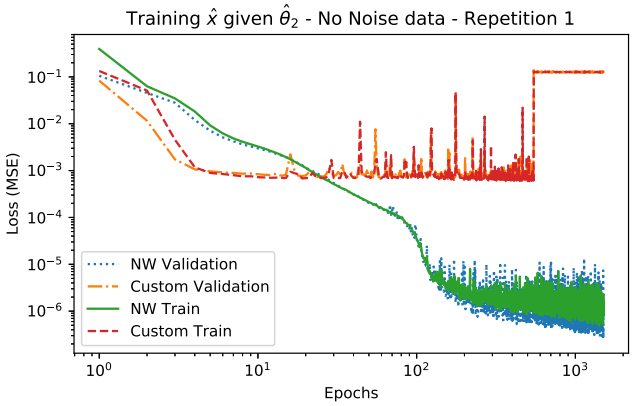

Fig. 13. History of errors with unbalanced weights during training

more inputs and solve the challenges we face in this process. One of it could be using for other two input combinations ($\hat{\theta}_1, \hat{\theta}_3$ and $\hat{\theta}_2, \hat{\theta}_3$) and three input combination ($\hat{\theta}_1, \hat{\theta}_2, \hat{\theta}_3$). One of the challenges working with more inputs or with more layers is concatenating multiple prototypes. Concatenating prototypes results in exploding weights. Balancing weights requires careful correspondence between the weights in the prototypes and the weights across the prototypes. So, our next steps include working on weight balancing techniques that helps to concatenate prototypes which is crucial for learning more deeper neural networks.

References

[1] G. Cybenko, "Approximation by superpositions of sigmoidal function," *Mathematics of Control, Signals, and Systems*, vol. 2, pp. 303–314, 1989.

[2] C. Rudin, "Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead," *Nature Medicine Intelligence*, vol. 1, pp. 206–215, 2019.

[3] J. S. Pei, "Parametric and nonparametric identification of nonlinear systems," Ph.D. Dissertation, Columbia University, 2001.

[4] J. S. Pei, J. P. Wright, and A. W. Smyth, "Mapping polynomial fitting into feedforward neural networks for modeling nonlinear dynamic systems and beyond," *Computer Methods in Applied Mechanics and Engineering*, vol. 194, no. 42-44, pp. 4481–4505, 2005.

[5] J. S. Pei and E. C. Mai, "Constructing multilayer feedforward neural networks to approximate nonlinear functions in engineering mechanics applications," *ASME Journal of Applied Mechanics*, vol. 75, 2008.

[6] J. S. Pei, E. C. Mai, and J. P. Wright, "Mapping some functions and four arithmetic operations to multilayer feedforward neural networks," in *SPIE International Symposia Smart Structures & Materials/NDE*, 2008.

[7] J. S. Pei, D. F. Hougen, S. T. Kanneganti, J. P. Wright, E. C. Mai, A. W. Smyth, S. F. Masri, A. Derkevorkian, F. Gay-Balmaz, and L. Komini, "Interpretable machine learning for function approximation in structural health monitoring," in *Structural Health Monitoring Based on Data Science Techniques*, A. Cury, D. Riberiro, F. Ubertini, and M. D. Todd, Eds. Springer, 2021, in press.

[8] D. F. Hougen, J.-S. Pei, and S. T. Kanneganti, "Toward interpretable machine learning for understanding epidemic data," in *2020 IEEE International Conference on Big Data; IEEE International Workshop on Fair and Interpretable Learning Algorithms*, Dec. 2020, pp. 3677–3681.

[9] K. Hauser, "Robotic systems," 2020. [Online]. Available: https://motion.cs.illinois.edu/RoboticSystems/

[10] J. Makhoul, A. El-Jaroudi, and R. Schwartz, "Formation of disconnected decision regions with a single hidden layer," in *International Joint Conference on Neural Networks*, vol. I, June 1989, pp. 455–460.

[11] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *Nature*, vol. 521, pp. 436–444, 2015.

[12] D. Gunning, M. Stefik, J. Choi, T. Miller, S. Stumpf, and G.-Z. Yang, "XAI - explainable artificial intelligence," *Science Robotics*, vol. 4, no. 37, p. eaay7120, 2019.

[13] E. D. Sontag, "Feedforward nets or interpolation and classification," *Journal of Computer and System Sciences*, vol. 45, pp. 20–48, 1992.

[14] J. S. Pei, E. C. Mai, J. P. Wright, and S. F. Masri, "Mapping some functions and four arithmetic operations to multilayer feedforward neural networks," *Nonlinear Dynamics*, vol. 71, no. 1-2, pp. 371–399, 2013.

[15] D. Nguyen and B. Widrow, "Improving the learning speed of 2-layer neural networks by choosing initial values of the adaptive weights," in *1990 International Joint Conference on Neural Networks*. IEEE, 1990, pp. 21–26.

[16] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," 2017.

[17] J. J. Moré, "The Levenberg-Marquardt algorithm: Implementation and theory," in *Numerical analysis*. Springer, 1978, pp. 105–116.

[18] P. E. McKnight and J. Najab, "Mann-Whitney U test," in *The Corsini Encyclopedia of Psychology*. American Cancer Society, 2010.

Authorized licensed use limited to: Universita degli Studi di Roma La Sapienza. Downloaded on July 30,2024 at 09:05:55 UTC from IEEE Xplore. Restrictions apply.